Classification

MADRE: Méthodologie pour l'Analyse de Données Réseaux
(Methodology for Network Data Analysis)

Francesca Fossati

2025-2026

Outline

1. Introduction
2. K-Nearest Neighbours classifier (KNN)
3. Logistic regression
4. Brief overview on other models
5. Model Evaluation Metrics

Classification problem

In machine learning, a classification problem consists of assigning input data to one of several predefined categories or classes.
Formally, given a dataset of $D_n = \{(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)\}$:
where:

- $x_i \in \mathbb{R}^K$ is a feature vector, as a vector of size K,
- $y_i \in \{1, 2, \ldots, M\}$ is the class label,

the goal is to learn a function $f : \mathbb{R}^K \to \{1, 2, \ldots, M\}$ that predicts the correct class for new, unseen inputs.
Classification can be:

- Binary ($M = 2$)
- Multiclass ($M > 2$)

Example: IoT classifier

Example application: Internet-of-Things (IoT) for home networks.

Home devices can be controlled from distance. (Camera, Light, Sensor, Mobile, Switch, Alarm, Tablet, Speaker, TV.)

For better quality-of-service these devices need to be identified by type from the network.

Massive number of devices with heterogeneous functionality

Use supervised learning to train an object classifier.

Example: classifying video QoE

How to detect video streaming QoE issues from encrypted traffic?

Use predictive models to detect different levels of QoE degradation, due to: stalling, average video quality, quality variations.

Ideal Bayes classifier

If all misclassifications are equally important,
An optimal classifier assign each observation to the most likely class, given its predictor values:

$$\max_{1 \leq j \leq M} Pr(Y = j \mid X = x_o)$$

We consider conditional probabilities given the observed x_o.

In a two-class problem: $Pr(Y = 1 \mid X = x_o) + Pr(Y = 2 \mid X = x_o) = 1$
Class 1, if $Pr(Y = 1 \mid X = x_o) > 0,5$
Class 2, if $Pr(Y = 2 \mid X = x_o) > 0,5$
Decision boundary $Pr(Y = 1 \mid X = x_o) = Pr(Y = 2 \mid X = x_o)$

Bayes example

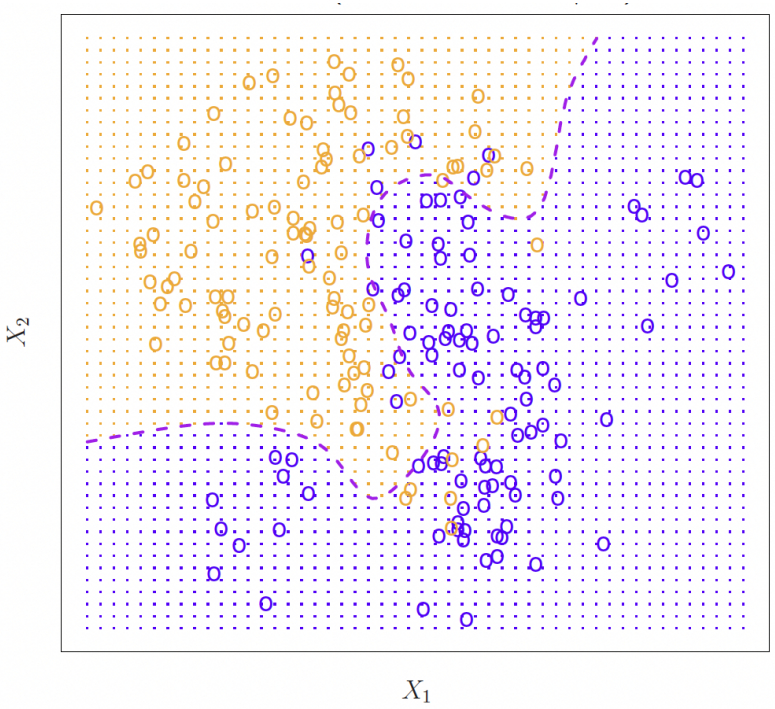


Figure: Bayes classifier[1]

[1] Gareth James et al. "An introduction to statistical learning: with applications in R". Springer Texts in Statistics.

Drawbacks

Circles that fall in regions with different colour: misclassifications

Bayes classifier produces lowest test error rate (irreducible) !

$$\text{Test Error}(x_o) = 1 - \max_j Pr(Y = j \mid X = x_o)$$

BUT! For real data we do not know the conditional distribution $P(Y \mid X)$, unless we have generated data ourselves, in which case we know the joint distribution.

Bayes classifier serves as an unreachable golden standard!

If we do not know exactly $P(Y \mid X)$ we can try to estimate it.

Outline

1. Introduction
2. K-Nearest Neighbours classifier (KNN)
3. Logistic regression
4. Brief overview on other models
5. Model Evaluation Metrics

KNN classifier

- Choose a positive integer $K > 0$[2].
- Given a test observation $x_o \notin D_n$, the KNN classifier identifies the K points in the training data-set closest to x_o, it is the set $\mathcal{N}_K(x_o)$.
- The conditional probability for class j at x_o is estimated as:

$$Pr(Y = j \mid X = x_o) = \frac{1}{K} \sum_{i \in \mathcal{N}_K(x_o)}^{n} \mathbb{1}(y_i = j).$$

- Calculate the estimates for all classes $j = 1, ..., M$ and
- Finally, apply Bayes classification: classify x_o to the class with the largest estimated probability.

[2] K is not the feauture vector size here

Optimal choice of K

Despite its simplicity KNN can give classifiers surprisingly close to Bayes.

Choice of K is important:

- If K $= 1$, very flexible decision boundary. Low Training Error ($= 0$) but High Test Error.
- As K increases (less flexibility). Training Error increases but the Test Error may not
- Find optimal K^* with minimum Test Error (U shape)
- If $K = n$ decision boundary close to linear.

Variance vs Bias Tradeoff or Flexibility vs Interpretability

Example

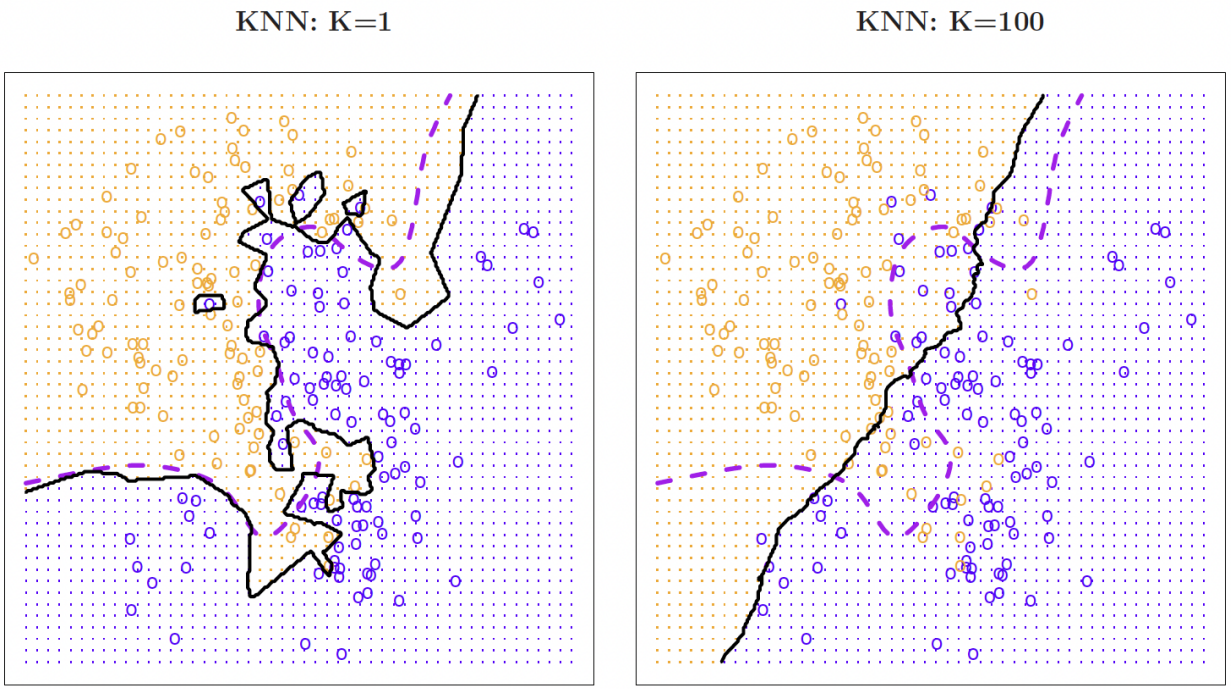


Figure: KNN with K=1 and K=100[3].

[3] Gareth James et al. "An introduction to statistical learning: with applications in R". Springer Texts in Statistics.

Example

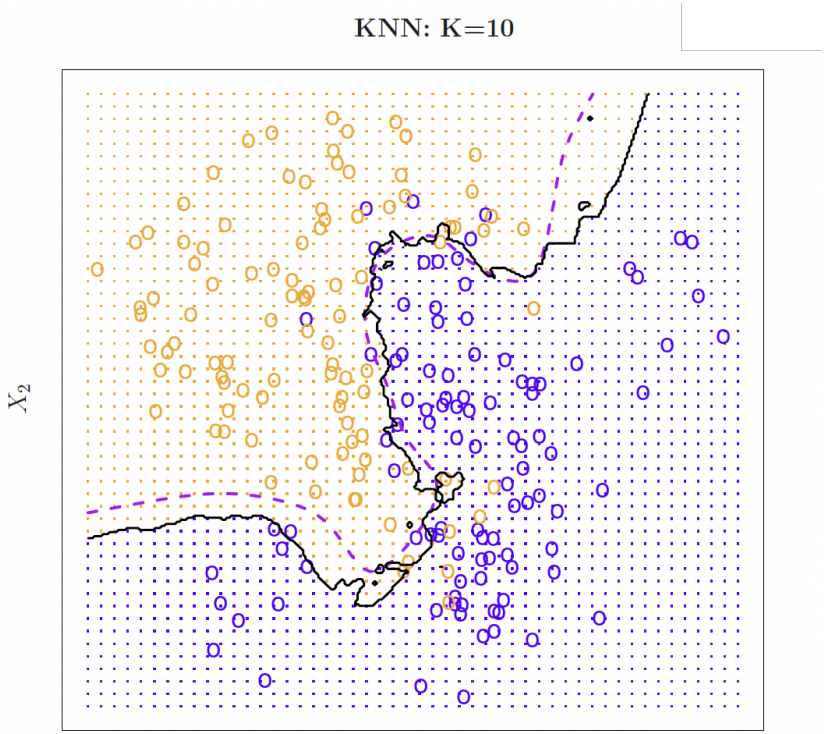


Figure: KNN with K=10. Close to Bayes optimal[4].

[4] Gareth James et al. "An introduction to statistical learning: with applications in R". Springer Texts in Statistics.

Variance vs Bias Tradeoff

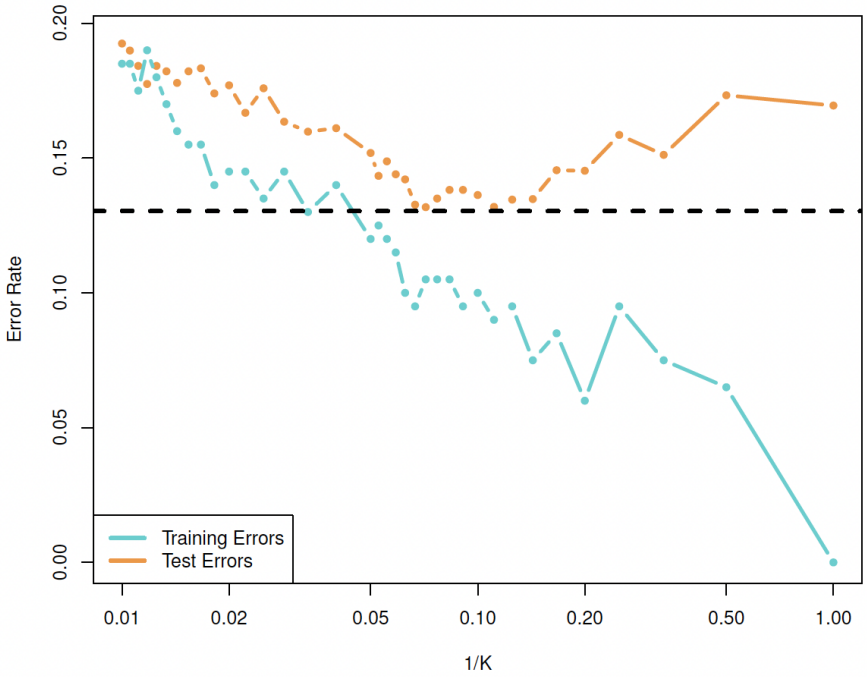


Figure: Training/Test error rate[5].

[5] Gareth James et al. "An introduction to statistical learning: with applications in R". Springer Texts in Statistics.

Pros and cons

Pros	**Cons**
Simple and intuitive to understand and implement.	Computationally expensive during prediction.
No training phase (lazy learner).	Sensitive to irrelevant features and feature scaling.
Non-parametric: no assumption about data distribution.	Suffers from the curse of dimensionality.
Can handle non-linear decision boundaries.	Requires entire training set in memory.
	No interpretability of the prediction logic.

Outline

1. Introduction
2. K-Nearest Neighbours classifier (KNN)
3. Logistic regression
4. Brief overview on other models
5. Model Evaluation Metrics

Linear regression

Suppose we have again two classes: 'Class 1', 'Class 2' and K = 1 feature.

- What if we used Linear Regression for the$P(Y \mid X)$?
- Let 'Class 1': Y = 0 and 'Class 2': Y = 1.
- We assume that the linear model describes the 0/1 data,
  $y_t = \beta_0 + \beta_1 x_1 + \epsilon$
  and we look for the regression line
  $\mathbb{E}[Y \mid X] = \hat{\beta}_0 + \hat{\beta}_1 X$

Since $Y_t \in \{0, 1\}$ then $\mathbb{E}[Y_t \mid X_t] = Pr(Y_t = 1 \mid X_t)) = \hat{\beta}_0 + \hat{\beta}_1 X_t$

Wrong shape

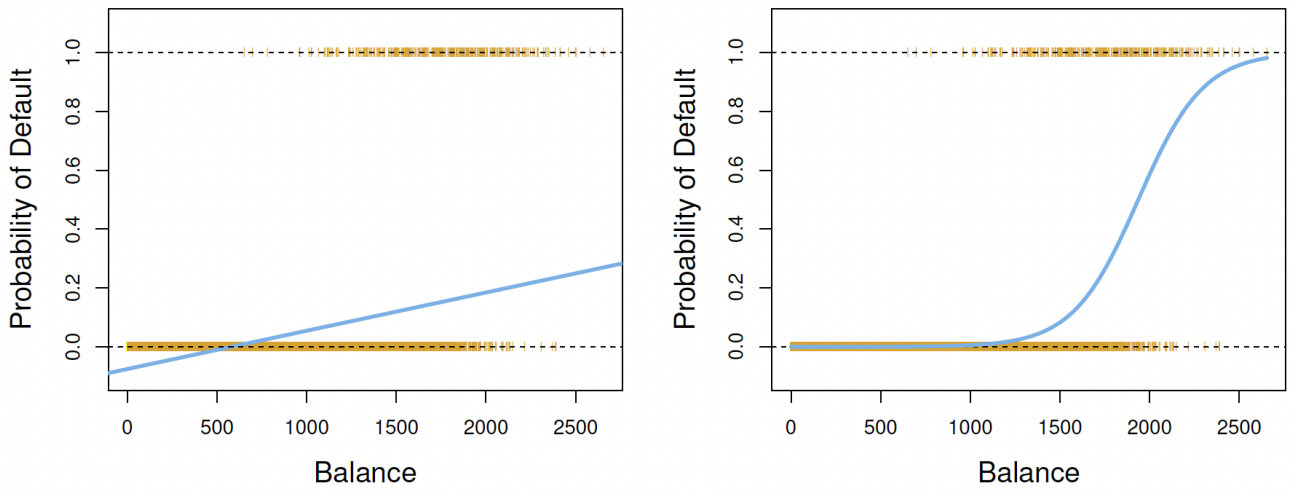


Figure: Linear vs Sigmoidal fit[6].

[6] Gareth James et al. "An introduction to statistical learning: with applications in R". Springer Texts in Statistics.

Logistic regression

Suppose for the two-class problem $Pr(Y = 1 \mid X)$ follows the logistic function.

$$p(X) = Pr(Y = 1 \mid X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}$$

- For $X \to -\infty : p(X) \to 0$
- For $X \to +\infty : p(X) \to 1$
- It is an S-shaped curve.

We need to fit β_0, β_1 in the non-linear logistic function.

Logistic fit

We consider a Training data-set D_n with $Y_n = (0, 0, 1, ..., 0, 1)$.

- We don't want to use MSE fit (complicated expressions).
- Better to use thr log-likelihood function.

What is the likelihood $g(D_n)$ of the data-sample?

$$g(D_n) = \prod_{t:y_t=1} p(x_t) \prod_{t':y'_t=0} (1 - p(x_{t'}))$$

because we assumed that for any X

$$Y = \begin{cases} 1, & p(X) \\ 0, & 1 - p(X) \end{cases}$$

and for all $x_t \in D_n$ we know what is the y_t answer.

Log-likelihood maximization

The log-likelihood function, is then equal to

$$\ell(\beta_0, \beta_1; D_n) = log(g(D_n)) =$$

$$\sum_{t:y_t=1} \log(p(x_t)) + \sum_{t':y'_t=0} \log(1 - p(x'_t)) =$$

$$\sum_{t=1}^{n} [y_t \log(p(x_t)) + (1 - y_i) \log(1 - p(x_t))] =$$

$$\sum_{i=1}^{n} \left[y_i(\beta_0 + \beta_1 x_i) - \log(1 + e^{\beta_0 + \beta_1 x_i}) \right]$$

We want to $\max_{\beta_0, \beta_1} \ell(\beta_0, \beta_1; D_n)$

Newton's method

- $\nabla \ell(\beta_0, \beta_1; D_n)$ is the gradient (first derivative) of the log-likelihood
- $\nabla^2 \ell(\beta_0, \beta_1; D_n)$ is the Hessian matrix (second derivative)

The Hessian matrix is negative-definite hence the log-likelihood function is strictly concave.

$$\begin{bmatrix} \beta_0^{(k+1)} \\ \beta_1^{(k+1)} \end{bmatrix} = \begin{bmatrix} \beta_0^{(k)} \\ \beta_1^{(k)} \end{bmatrix} - (\nabla^2 \ell(\beta_0, \beta_1; D_n))^{-1} \nabla \ell(\beta_0, \beta_1; D_n)$$

The odds

One can see the logistic expression of the predictions from a different point-of-view:

$$q(x_t) := \frac{p(x_t)}{1 - p(x_t)} = e^{(\beta_0 + \beta_1 x_t)}$$

Odds function: often used in... Horse-racing!
"What are the odds ?"

- if $q(x_t) = 1/4$ then $p(x_t) = P(Y_t = 1 \mid x_t) = 0.2$
- if $q(x'_t) = 9/1$ then $p(x'_t) = P(Y_t = 1 \mid x'_t) = 0.9$

The logits

$$Q(x_t) := log\left(\frac{p(x_t)}{1-p(x_t)}\right) = \beta_0 + \beta_1 x_t$$

Here we come back to the expression for the Linear Regression!

Separating hyperplane: For $p = 0,5$, we get the "linear" boundary

$$0 = \beta_0 + \beta_1 x_{t,1}(+\beta_2 x_{t,2} + ...\beta_d x_{t,K}), \text{ for } K \geq 1$$

e.g. for $K = 1$, it is a point $x_{bound} = -\beta_0/\beta_1$.

Example

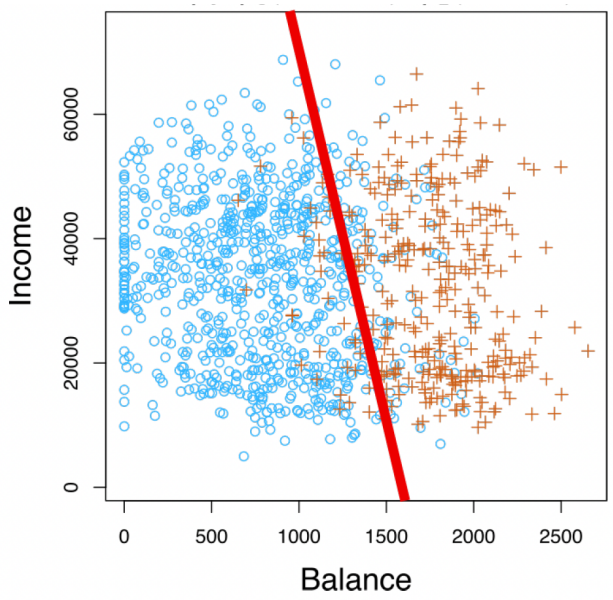


Figure: The boundary separates "blue" from "orange": $\beta_0 + \beta_1(Balance) + \beta_2(Income) = 0$[7]

[7] Gareth James et al. "An introduction to statistical learning: with applications in R". Springer Texts in Statistics.

Test data

If we have test input data $x_o \in D_n$, how do we choose its Class?
Say $x_o = (x_{o,1}, x_{o,2}, ..., x_{o,K})$.
Use the fitted values of $\beta_0, \beta_1, ..., \beta_K$

- Either calculate $p(x_o)$ and check if $>, =, < 0.5$
- or check the position of x_o related to the boundary:

$$\beta_0 + \beta_1 x_{o,1} + \beta_2 x_{o,2} + ...\beta_D x_{o,K} >, =, < 0$$

We need not always use the value of 0,5 for the boundary...

Multiple logistic regression

We have implied that the Logistic Regression is generalised to higher than 1 dimension:

$$log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 X_1 + ... + \beta_1 X_K$$

where $X = (X_1, ..., X_K)$ are D predictors. Equivalently,

$$p(X) = \frac{e^{(\beta_0+\beta_1 X_1+...+\beta_K X_K)}}{1+e^{(\beta_0+\beta_1 X_1+...+\beta_K X_K)}}$$

$\beta_0, \beta_1, ...\beta_K$ are estimated by the maximum likelihood method.

Logistic regression for > 2 classes

We can easily generalise to M classes:

$$\log \frac{Pr(Class = 1 \mid X = x)}{Pr(Class = M \mid X = x)} = \beta_{1,0} + \beta_1^T x$$

$$\dots$$

$$\log \frac{Pr(Class = M-1 \mid X = x)}{Pr(Class = M \mid X = x)} = \beta_{M-1,0} + \beta_{M-1}^T x$$

$$Pr(Class = M \mid X = x) = \frac{1}{1 + \sum_{m=1}^{M-1} exp(\beta_{m,0} + \beta_m^T x)}$$

We need $M-1$ log-odds, The probabilities sum-up to 1, The choice of denominator class is arbitrary, Max likelihood.
For multiple classes, discriminant analysis is more popular...

Pros and cons

Pros	**Cons**
Fast to train and predict.	Assumes linear relationship between features and log-odds.
Outputs calibrated probabilities.	Not flexible for non-linear relationships (without feature engineering).
Model is interpretable via feature coefficients.	Sensitive to outliers.
Works well with high-dimensional data.	Requires meaningful feature engineering.
Has a strong theoretical foundation.	

Outline

1. Introduction
2. K-Nearest Neighbours classifier (KNN)
3. Logistic regression
4. Brief overview on other models
5. Model Evaluation Metrics

Outline

- Decision tree
- Random forest
- Support vector machines (SVM)
- Neural networks for classification

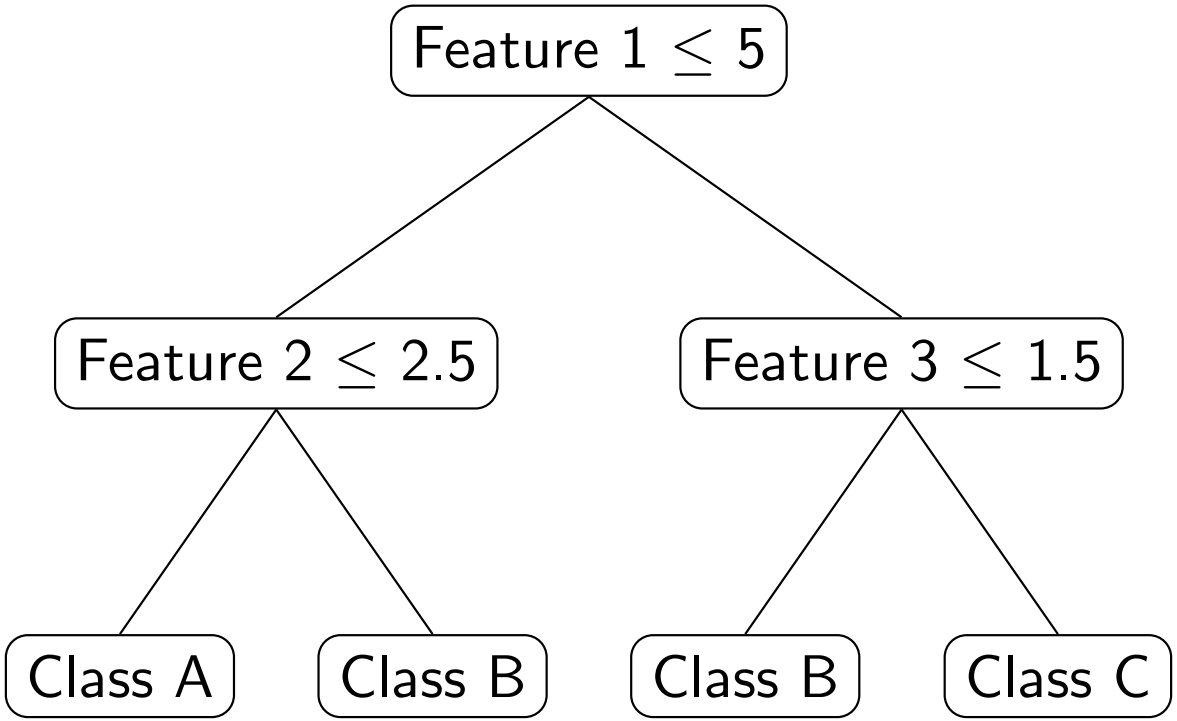
Decision tree

A decision tree is recursively splits the feature space based on feature values to create a tree-like model of decisions.

Each internal node represents a decision based on a feature, each branch represents an outcome of the decision, and each leaf node corresponds to a class label.

At each node, the algorithm selects the best feature and threshold to split the data in order to increase the purity of the resulting subsets.

Example


Feature 1 ≤ 5
Feature 2 ≤ 2.5
Feature 3 ≤ 1.5
Class A
Class B
Class B
Class C

Random forest

A Random forest operates by constructing a multitude of decision trees during training time and outputting the mode (most common class) of the classes predicted by individual trees for classification tasks.

It is based on the idea of combining the predictions of multiple weak learners (decision trees) to produce a more accurate and robust model.

Random Forest uses the bagging technique to train each decision tree on a different random subset of the training data. Each subset is created by sampling with replacement from the original dataset.

At each split in a tree, instead of considering all features, a random subset of features is chosen. This increases the diversity of the trees and reduces correlation between them.

Support vector machine (SVM)

Support Vector Machine (SVM) aims to find the optimal hyperplane that separates data from different classes with the largest possible margin.

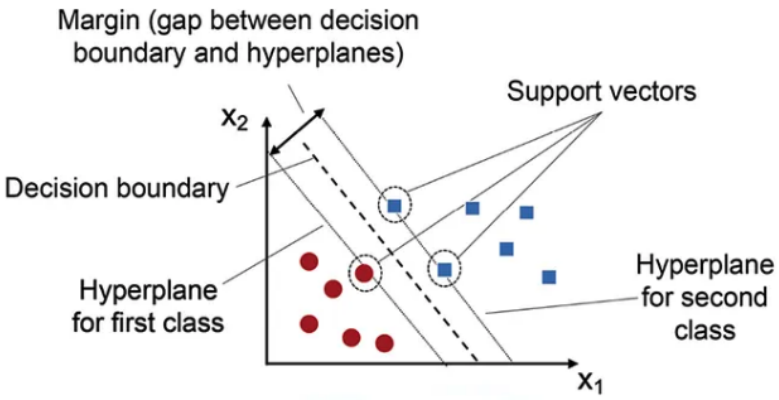
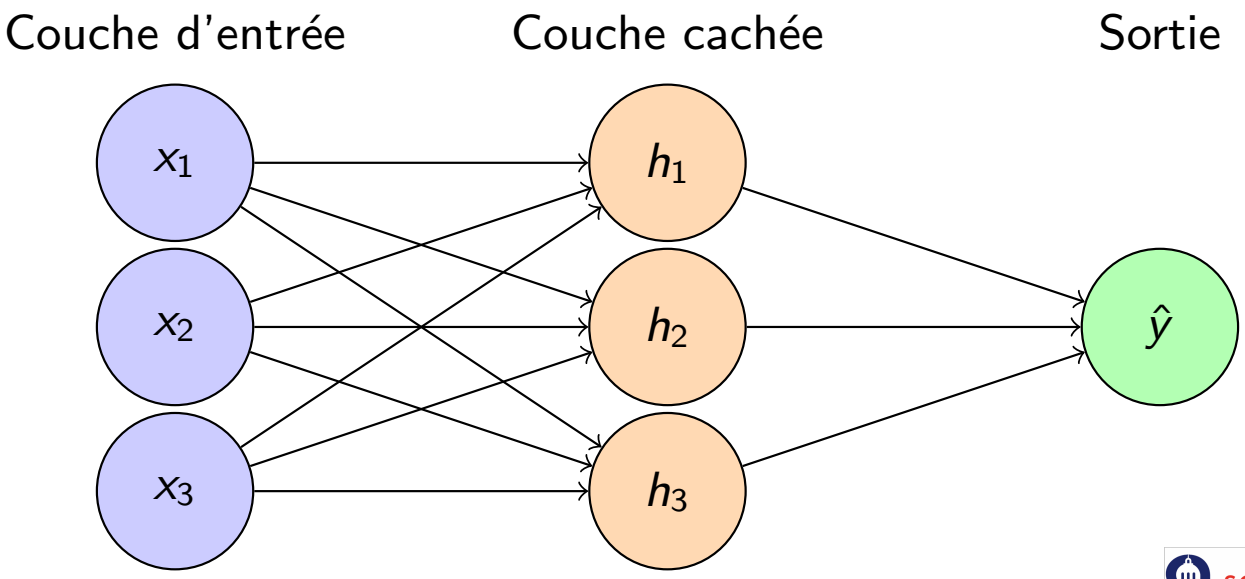


Figure source: https://medium.com/low-code-for-advanced-data-science/support-vector-machines-svm-an-intuitive-explanation-b084d6238106

Neural networks for classification

A neural network (NN) is a supervised learning model inspired by the human brain. It is composed of layers of interconnected artificial neurons. Each neuron performs a transformation of its inputs followed by an activation function.



Pros and cons

Model	Advantages	Disadvantages
Decision Tree	Easy to interpret, no need for feature scaling, handles categorical data	Prone to overfitting, unstable to small data changes
Random Forest	Reduces overfitting, robust, good bias-variance trade-off	Less interpretable, slower to train
SVM	Effective in high dimensions, works well with clear margins	Sensitive to parameter choice, not ideal for large datasets
Neural Network	Very powerful, learns feature representations, handles large data volumes	Requires lots of data, less interpretable, computationally expensive

Outline

1. Introduction
2. K-Nearest Neighbours classifier (KNN)
3. Logistic regression
4. Brief overview on other models
5. Model Evaluation Metrics

Evaluation metrics

- Test accurancy
- Confusion matrix
- Precision and recall
- Scecificity
- ROC Curve

Training Accurancy

Suppose we have training observations:
$D_n = \{(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)\}$, with $y_1, ..., y_n$ qualitative.
Consider a fitting model with an estimate $\hat{y}_i = \hat{f}(x_i)$.
The training error rate:

$$\frac{1}{n} \sum_{i=1}^{n} \mathbb{1}(y_i \neq \hat{y}_i)$$

is the fraction of incorrect classifications:

- $\hat{y}_i$ is the predicted class label for the i-th observation using $\hat{f}$.
- $\mathbb{1}(y_i \neq \hat{y}_i) = 0$ for correct classification, else 1.
- Similar to MSE_{train} in regression!

Test Accurancy

Most interested in the error rates of the classifier to test observations $(x_o, y_o) \notin D_n$, not used in training.
Again for an estimate $\hat{y}_o = \hat{f}(x_o)$ we use the test error rate:

$$Ave(\mathbb{1}(y_o \neq \hat{y}_o))$$

A good classifier is the one for which the test error is smallest !

Confusion matrix

		Predicted condition	
	Total population = P + N	Positive (PP)	Negative (PN)
Actual condition	Positive (P)	True positive (TP)	False negative (FN)
	Negative (N)	False positive (FP)	True negative (TN)

Two types of errors:

- FP: Incorrectly assign an individual of Class N to Class P.
- FN: Incorrectly assign an individual of Class P to Class N.

Figure source: Wikipedia (Confusion matrix)

Accurancy

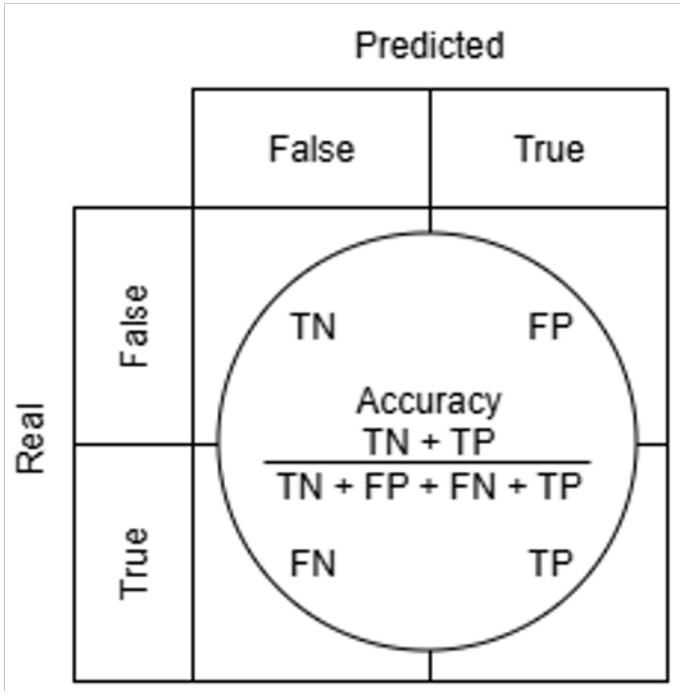


Figure source: Wikipedia (Confusion matrix)

Precision

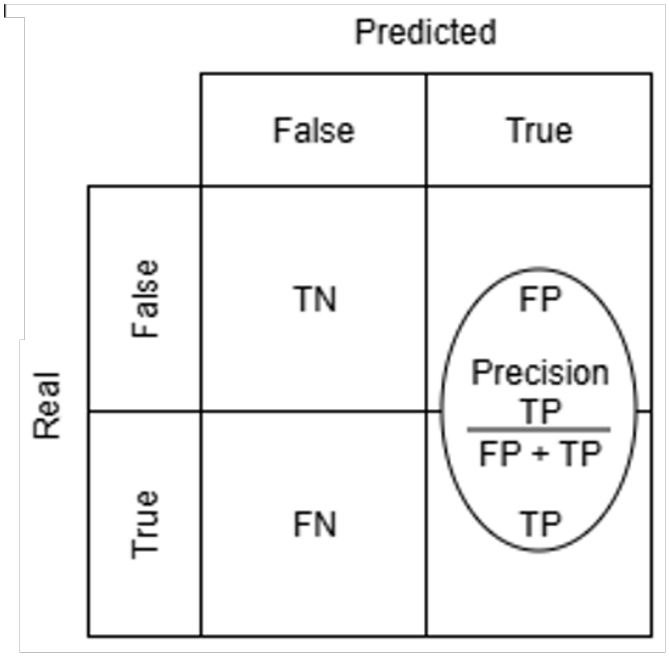
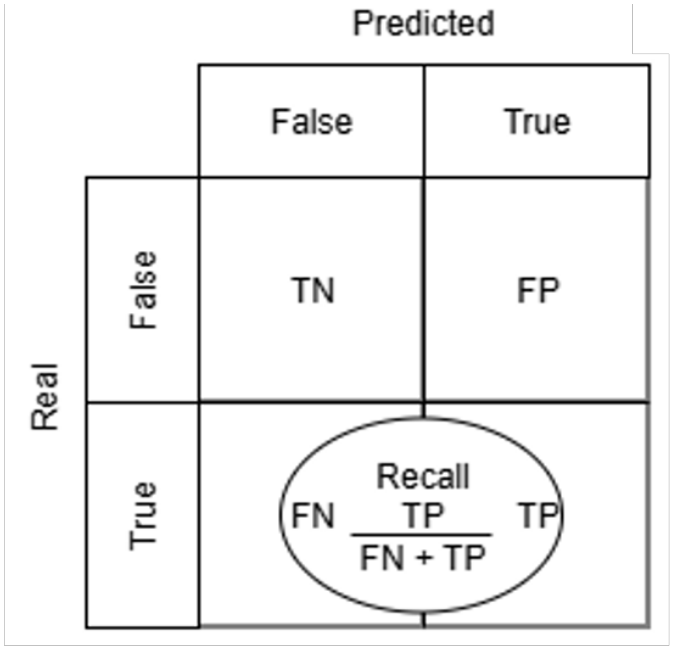


Figure source: Wikipedia (Confusion matrix)

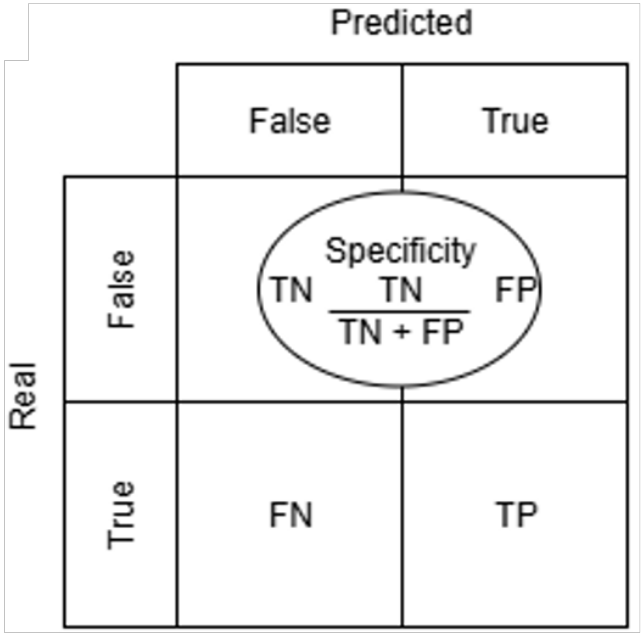
Recall



False negative rate= 1-Recall

Figure source: Wikipedia (Confusion matrix)

Specificity



False positive rate= 1-Specificity

Figure source: Wikipedia (Confusion matrix)

F1 score

$$F1 = 2 \cdot \frac{Precision + Recall}{Precision \cdot Recall}$$

It is an harmonic mean of precision and recall.
It is good for imbalanced classes.

ROC curve

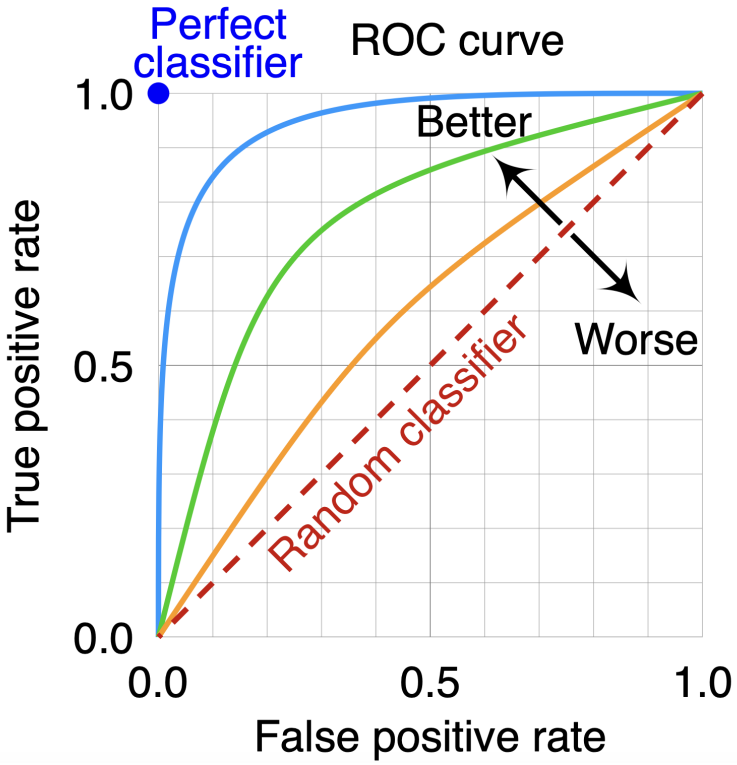


Figure source: Wikipedia (Receiver Operating Characteristic)

Exercice

A binary classifier is trained to detect whether emails are spam or not spam. After testing the model on 200 emails, the following confusion matrix was obtained:

	Predicted: Spam	Predicted: Not Spam
Actual: Spam	50	10
Actual: Not Spam	30	110

- What is the accuracy of the model?
- What is the precision for the "spam" class?
- What is the recall (sensitivity) for the "spam" class?
- What is the specificity (true negative rate for not spam)?
- What is F1 score?